

ASPA – приложение ASP.NET CORE

♣ – задания, требующие самостоятельного изучения (нет в лекциях)
Используйте библиотеку **DAL_Celebrity_MSSQL** разработанную в предыдущем задании.

Оформление html-разметки на усмотрение разработчика

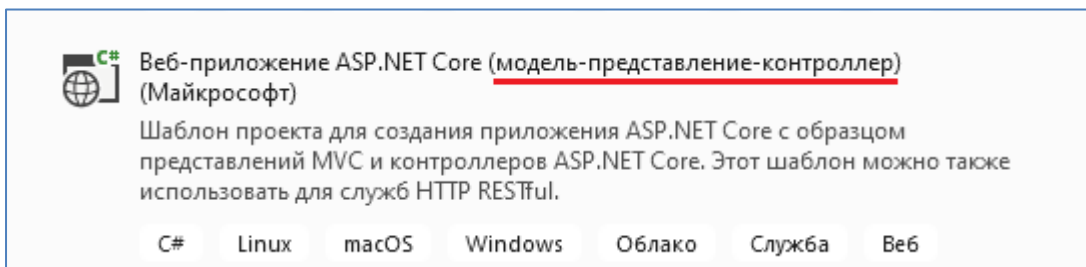
Задание 1.

1. Разработайте ASPA, применив следующий шаблон.

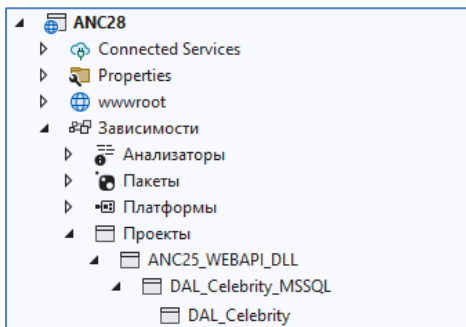
Имя решения: **ASPA**

Имя проекта: **ASPA008_1**

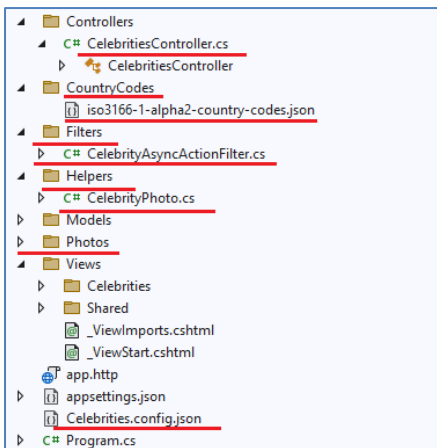
Версия .NET: **7.0** или **8.0**



2. Структура проекта приведена и пояснена ниже



ANC25_WEBAPI_DLL	Библиотека, содержащая - сервисы: IRepository (репозиторий БД Celebrities) IOptions<CelebritiesConfig> (конфигурация), CountryCodes (ISO-коды стран), CelebrityTitles (заголовки представлений); - конечные точки: MapGet, MapPost, MapPut, MapDelete - разработанные в предыдущих лабораторных работах. - middleware-обработчик ошибок MiddlewareErrorHandler
Dal Celebrity MSSQL	Библиотека, разработанная в предыдущих лабораторных работах
Dal_Celebrity_	Библиотека, разработанная в предыдущих лабораторных работах



ContryCodes	Директорий с json-файлом с кодами стран (прилагается)
Controllers	Контроллер CelebritiesController, выполняющий: - отображение фотографий всех знаменитостей; - отображение информации выбранной знаменитости; - добавление информации о новой (с подтверждением) знаменитости; - удаление информации о существующей знаменитости (с подтверждением).
Filters	Action-фильтр для метода котроллера (добавление ссылок о знаменитости из сервиса Wikipedia), отображающего знаменитость
Helpers	HTML-helper CelebrityFoto для отображение фотографии знаменитости
Photos	Директорий с файлами-фотографиями знаменитостей.
Celebrities.config.json	Конфигурационный файл приложения

3. Примерный вид файла Program.cs

```
private static void Main(string[] args)
{
    var builder = WebApplication.CreateBuilder(args);

    builder.AddCelebritiesConfiguration(); // подключение конфигурации IOptions<CelebritiesConfig>

    builder.AddCelebritiesServices(); // подключение сервисов: IRepository, CountryCodes, CelerbrityTitles

    builder.Services.AddControllersWithViews();
    var app = builder.Build();
    app.UseHttpsRedirection();
    if (!app.Environment.IsDevelopment()) {app.UseExceptionHandler("/Home/Error");}
    app.UseStaticFiles();
    app.UseRouting();

    app.UseCelerbritiesErrorHandler("ANC28"); // подключение обработчика ошибок
    app.MapCelebrities(); // Map для Celebritirs
    app.MapLifeevents(); // Map для Lifeevents
    app.MapPhotoCelebrities(); // Map для фотографий

    app.UseAuthorization();

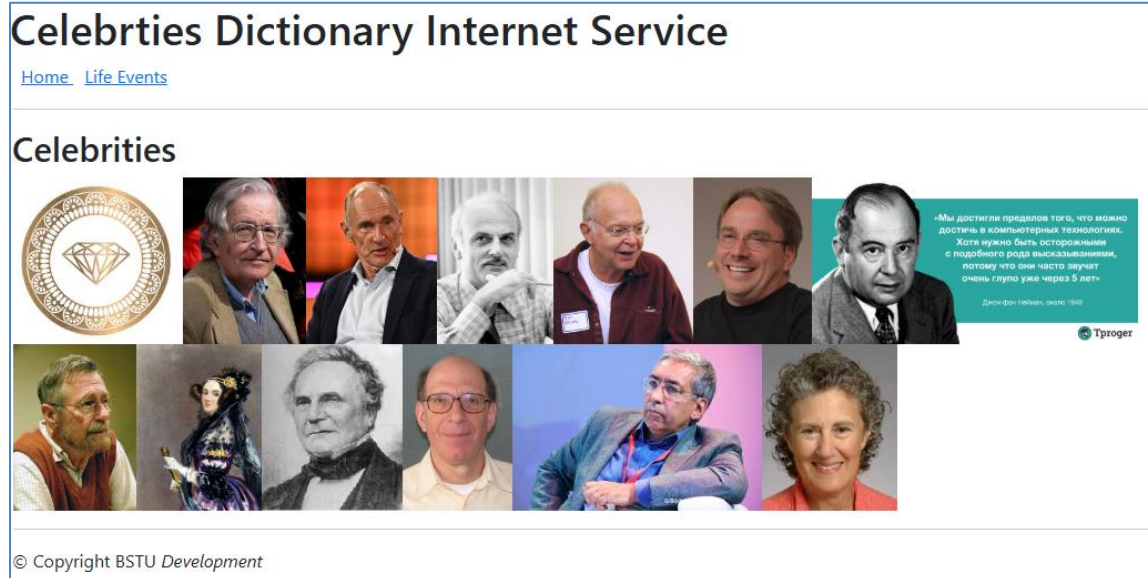
    // маршрутизация
    app.MapControllerRoute( // добавление новой знаменитости
        name: "celebrity",
        pattern: "/0",
        defaults: new { Controller = "Celebrities", Action = "NewHumanForm" });
    app.MapControllerRoute( // дейсвия с одной знаменитостью
        name: "celebrity",
        pattern:("/{id:int:min(1)}",
        defaults: new {Controller = "Celebrities", Action = "Human"});
    app.MapControllerRoute( // отображение всех зменитостей или одной (id)
        name: "default",
        pattern: "{controller=Celebrities}/{action=Index}/{id?}");

    app.Run();
}
```

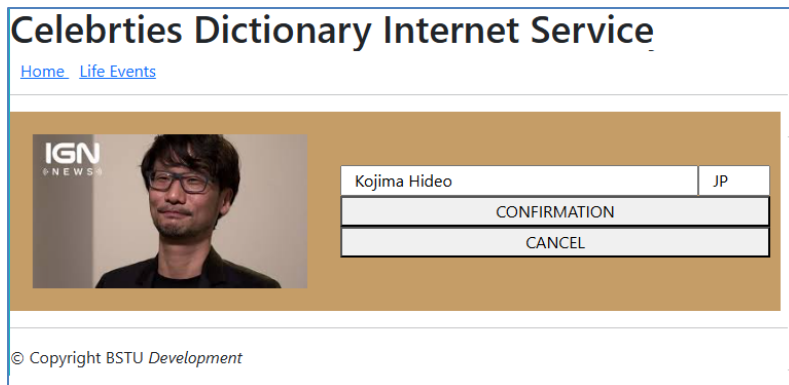
4. Примерный вид файла конфигурации

```
{
  // Celebrities.config.json
  "Celebrities": {
    "PhotosRequestPath": "/Photos",
    "PhotosFolder": "D:\\VS_Projects\\ANC\\ANC28\\Photos",
    "ConnectionString": "Data source=172.16.193.88; Initial Catalog=LES28X; TrustServerCertificate=True; User Id=sa; Password=123456",
    "ISO3166alpha2Path": "D:\\VS_Projects\\ANC\\ANC28\\CountryCodes\\iso3166-1-alpha2-country-codes.json"
  }
}
```

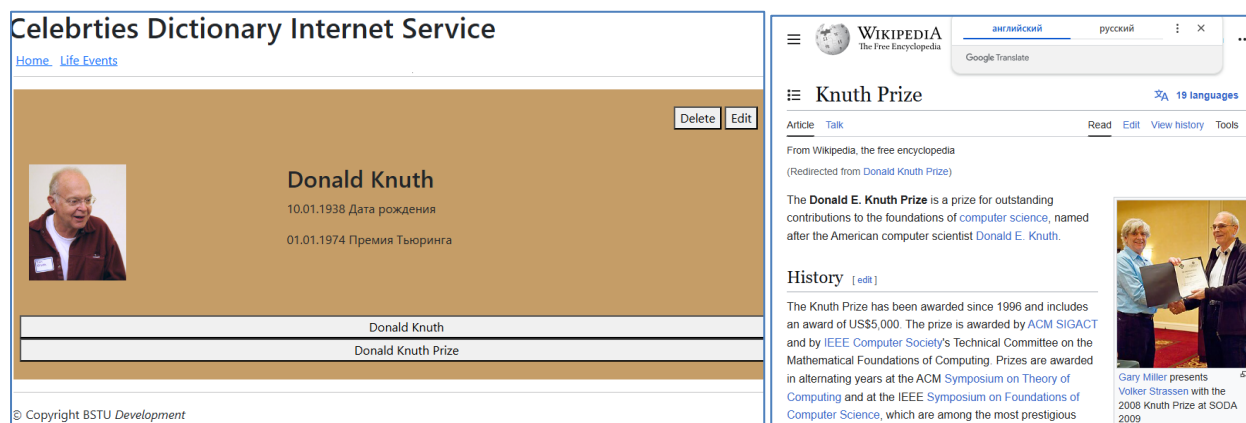
5. Примерный вид приложения при запуске



6. Примерный вид форм (ввод и подтверждение) для ввода информации о новой знаменитости (активизируется при клике мышью на **бриллиант**). Обратите внимание, при выборе поля национальной принадлежности должен применяться dropdownlist-элемент (ContryCodes).



7. Примерный вид отображения информации о знаменитости (активизируется при клике мышью на фотографию. Клавиша Edit – для корректировки, Delete – для удаления. Клавиши с именем знаменитости (внизу) формируются динамически в Action-фильтре, и предназначены для перехода на статьи Wikipedia об знаменитости.



8. Примерный вид программного кода Action-фильтра

```
public class InfoAsyncActionFilter : Attribute, IAsyncActionFilter
{
    public static readonly string Wikipedia = "WIKI";
    public static readonly string Facebook = "FACE";
    string infotype;
    public InfoAsyncActionFilter(string infotype = "") { this.infotype = infotype.ToUpper(); }
    public async Task OnActionExecutionAsync(ActionExecutingContext context, ActionExecutionDelegate next)
    {
        IRepository? repo = context.HttpContext.RequestServices.GetService<IRepository>();
        int id = (int)(context.ActionArguments["id"] ?? -1);
        Celebrity? celebrity = repo?.GetCelebrityById(id);
        if (infotype.Contains(Wikipedia) && celebrity != null)
            context.HttpContext.Items.Add(Wikipedia, await WikiInfoCelebrity.GetRefereces(celebrity.FullName));
        if (infotype.Contains(Facebook) && celebrity != null)
            context.HttpContext.Items.Add(Facebook, getFromFace(celebrity.FullName));
        await next();
    }
    string getFromFace(string fullname) { return "Info from Face"; }
}
```

```

public class WikiInfoCelebrity
{
    HttpClient client;
    //string wikiURITemplate = "https://en.wikipedia.org/w/api.php?action=opensearch&search=\"{0}\"&prop=info&format=json";
    Dictionary<string, string> wikiReferens { get; set; }
    string wikiURI;
    private WikiInfoCelebrity(String fullname)
    {
        this.client = new HttpClient();
        this.wikiReferens = new Dictionary<string, string>();
        // this.wikiURITemplate = fullname;
        this.wikiURI = string.Format("https://en.wikipedia.org/w/api.php?action=opensearch&search=\"{0}\"&prop=info&format=json", fullname);
    }
    public static async Task<Dictionary<string, string>> GetRefereces(string fullname)
    {
        WikiInfoCelebrity info = new WikiInfoCelebrity(fullname);
        HttpResponseMessage message = await info.client.GetAsync(info.wikiURI);
        if (message.StatusCode == System.Net.HttpStatusCode.OK)
        {
            List<dynamic>? result = await message.Content.ReadFromJsonAsync<List<dynamic>>() ?? default(List<dynamic>);
            List<string>? ls1 = JsonSerializer.Deserialize<List<string>>(result[1]);
            List<string>? ls3 = JsonSerializer.Deserialize<List<string>>(result[3]);
            for (int i = 0; i < ls1.Count; i++)
            {
                info.wikiReferens.Add(ls1[i], ls3[i]);
            }
        }
        return info.wikiReferens;
    }
}

```

9. Примерный вид формы для редактирования информации о знаменитости (активизируется при клике мышью на клавишу **Edit**) .

Celebrities Dictionary Internet Service

[Home](#) [Life Events](#)



Kaspersky	Russia
Kaspersky_tmpECE9.tmp.jpg	
OK	CANCEL

© Copyright BSTU Development

10. Примерный вид форм для подтверждения удаления информации знаменитости (активизируется при клике мышью на клавишу **Delete**) .

Celebrties Dictionary Internet Servi

[Home](#) [Life Events](#)



Linus Torvalds	US
DELETE CONFIRMATION	
CANCEL	

© Copyright BSTU Development

11. Для отображения фотографий используйте Html-helper. Примерный его код и пример использования приведен ниже.

```
public static class CelebrityHelpers
{
    public static HtmlString CelebrityPhoto (this IHtmlHelper html, int id, string title, string src)
    {
        string onclick = "location.href='/${this.id}'";
        string onload =
            "let h=this.height;let w=0;"+
            "let k=this.naturalWidth/this.naturalHeight;"+
            "if(h!=0 && w==0){this.height=h;this.width=k*h;}"+
            "if(h==0 && w!=0){this.height=w/k;this.width=k*h;}";
        string result = "<img id=\"{id}\" \" +
            \"$class=\"celebrity-photo\" \" +
            \"$title=\"{title}\" \" +
            \"$src=\"{src}\" \" +
            \"$onclick=\"{onclick}\" \" +
            \"$onload=\"{onload}\" \" +
            \"$\"/>";
        return new HtmlString(result);
    }
}
```

```
@using DAL_Celebrity_MSSQL;
@using ANC28.Helpers;
@model ANC28.Controllers.CelebritiesController.IndexModel
@{
    //Celebrities/Index.cshtml
    Layout = "_CelebritiesLayout";
}

@section subhead
{
    <h2>Celebrities</h2>
}

@Html.CelebrityPhoto(0,"New Celebrity",$"{Model.photosrequestpath}/{\"NewCelebrity.jpg\"}")
@foreach (Celebrity c in Model.celebrities) // по всем знаменитостям
{
    @Html.CelebrityPhoto(c.Id,c.FullName,$"{Model.photosrequestpath}/{c.ReqPhotoPath}")
}

@section foot
{
    @* <h2>FOOT</h2>*@
}
```

12. Используйте макет (Layout) при верстке представлений приложения. Примерный код макета представлен ниже.

```
@using static ANC25_WEBAPI_DLL.CelebritiesAPIExtensions;
@inject CelebrityTitles titles
@{
    //CelebritiesLayout.cshtml
}

<!DOCTYPE html>
<html>
<head>
    <meta name="viewport" content="width=device-width" />
    <title>@titles.Title</title>
    <link href="~/lib/bootstrap/dist/css/bootstrap.min.css" asp-append-version="true" rel="stylesheet" />
    <link href="~/css/Celebrities.css" asp-append-version="true" rel="stylesheet" />
    <script src="~/js/Celebrities.js" asp-append-version="true"></script>
</head>
<body>
    <script src="~/framework/aspnetcore-browser-refresh.js"></script>

    <h1>@titles.Head</h1>
    <div class="row">
        <span class="col-2">
            <a href="/" style="margin-left: 10px;"> Home </a> <a href="/" style="margin-left: 10px;"> Life Events </a>
        </span>
        <span class="col-4"></span>
        @* <form class="col-5" method="post">
            <input type="text" name="name" placeholder="@username" />
            <input type="password" name="pass" placeholder="password" />
            <button type="submit"> Sign In</button>
        </form>*@
    </div>
    <div>
        @RenderSection("subhead",false)
        @RenderBody()
        @RenderSection("foot",false)
    </div>
    <hr />
    &#169; @titles.Copyright
    <environment include="Development,Staging">
        <i>Development</i>
    </environment>
</body>
</html>
```


13. Для ввода данных о знаменитости, используйте частичное представление. Примените хелперы: `Html.BeginForm` (для создания формы) и `Html.DropDownList` (для ввода национальности). Для доступа к справочнику стран разработайте сервис и примените директиву **@inject** для внедрения сервиса в этом частичном представлении. Пример применения хелперов и сервиса приведен ниже.

```
@using Microsoft.AspNetCore.Mvc.Rendering
@inject ANC25_WEBAPI_DLL.CelebritiesAPIExtensions.CountryCodes Countries
@{
    // _InFormNewCelebrity
    SelectList slcountries = new SelectList(Countries, "code", "countryLabel");
}
@using (Html.BeginForm(FormMethod.Post, new { enctype = "multipart/form-data" }))
{
    <div class="row">
        @Html.TextBox("fullname", "", new { @class = "col-8", placeholder="Celebrities Name", required="true" })
        @Html.DropDownList("Nationality", slcountries, new { @class = "col-4" })
    </div>
    <div class="row">
        <input id="upload" name="upload" type="file" style="display:none;" accept="image/jpeg" onchange="onChange(this)" required />
        <button class="col-12" id="butupload" type="button" onclick="document.getElementById('upload').click()"> Upload Photo </button>
    </div>
    <div class="row">
        <button class="col-6" type="submit"> OK </button>
        <button class="col-6" type="reset"> CANCEL</button>
    </div>
}

<script>
    function onChange(e) {
        if (e.value == '') butupload.innerText = "Upload Photo";
        else butupload.innerText = e.value;
    }
</script>
```

14. Продемонстрируйте работоспособность приложения.